LOVE SIMULATOR: ESCAPE FROM EX

Chen Yang, Ziqi Lin, Lesi Kong

1 Introduction

This report details the design, implementation, and evaluation of LOVE SIMULATOR: ESCAPE FROM EX, an interactive text-based adventure game developed entirely in SWI-Prolog. The project serves as a practical application of knowledge representation and reasoning principles, demonstrating how symbolic AI paradigms can be used to create complex, dynamic systems. The game's core objective is to model the intricate relationship between spatial navigation, psychological stress, and intelligent adversary pursuit. Players assume the role of a protagonist navigating the difficult situation of still cohabiting with their ex-partner (EX) three days after a breakup. The primary goal is to collect a set of essential items including a key and a suitcase to unlock and use the exit, all while managing a constantly depleting mental health score and evading a sophisticated, PDDL-driven adversary. The game operates on a discrete turn-based system, where every player command—ranging from movement (go) to simple information queries (inventory, mental)—constitutes a turn, triggering a calculated response from the adversary. This design choice ensures a persistent sense of urgency and elevates the gameplay from a simple puzzle to a strategic challenge, a core pursuit in modern AI-driven game design for creating immersive player experiences [5].

Our conceptual inspiration for this project is drawn from the "Otome game" genre, a category of story-based video games that are targeted towards a female audience and typically focus on developing deep emotional and parasocial relationships with virtual male characters[2], with a prominent contemporary example being Love and Deepspace. Our project is conceived as a thematic inversion of this genre. We reasoned that if a significant portion of the gaming landscape is dedicated to teaching players how to form and nurture relationships, there exists a compelling narrative and mechanical space for games that explore the logic and emotional difficulty of escaping an unhealthy one. The fundamental difference between our game and its inspiration, therefore, lies in this thematic and mechanical opposition: where Love and Deepspace uses its systems to foster connection, our gameplay is architected around the mechanics of strategic evasion and severance. We achieve this through an expressive adversary system where the EX's behaviour is not rigidly scripted but evolves based on an explicit mapping between internal emotional states, strategic rules encoded in Prolog, and high-level goals defined in PDDL. By integrating Prolog's logical reasoning capabilities with PDDL's automated planning, we have constructed a complete, closed-loop system that cycles from knowledge-based state assessment to goal-oriented plan generation and execution.

2 Knowledge Representation in Prolog

Our game map and initial state diagram are shown in Figure 1.Our choice to use Prolog as the foundational technology for the game's knowledge base (KB) was a deliberate one, driven by the language's inherent strengths in symbolic reasoning and declarative programming. This approach allows us to define the game world's entities, properties, and logical constraints in a human-readable and formally precise manner, which is a key principle in the field of knowledge representation[1]. Our primary reasoning for this architecture was to build a system where the game's logic is transparent and easily extensible. The knowledge base is logically partitioned into static and dynamic facts. Static facts represent the immutable structure of the game world, including the set of rooms declared with room/1, the topological map defined by the bidirectional connected/3 predicate, and the initial placement of objects via item/2. This design choice strictly separates the world's physical structure from its transient state, ensuring environmental stability. All static facts are actively used, leaving no redundant knowledge. Conversely, dynamic facts, declared with :- dynamic/1, represent the game's mutable state. These include player_location/1, npc_location/1, inventory/1, mental/1, and the adversary's ex_mood/1. We chose to model these elements as dynamic because they are the direct inputs and outputs of actions, requiring updates via assert/1 and retract/1 each turn to keep the KB synchronized with the world. The game's core interactivity is governed by a comprehensive set of Prolog rules that operate on these facts. These rules implement robust precondition checks for all player commands to maintain world integrity, and they also define the game's four distinct termination conditions (escape, mental collapse, capture, reconciliation), which are unified under a single game_over(true) flag. To enhance strategic depth, we introduced randomness via random/3 to govern the adversary's fallback movement and capture probabilities. Furthermore, arithmetic functions are central to the mechanics, primarily for manipulating the mental value (e.g., NewM is M - 2), which directly links spatial location to the player's psychological state and narrative experience.

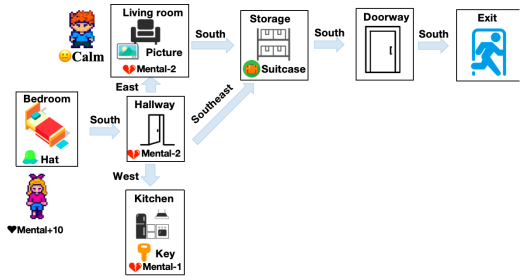


Figure 1: Game map and initial state diagram.

Game map and initial state diagram the game's spatial structure. The player starts in the bedroom with a mental value of 10, while the EX starts in the living_room in a calm state. The map contains seven rooms connected to form a path to the exit. Key items are distributed across the map, and certain rooms trigger mental value changes, forming the environmental basis for gameplay and adversary planning.

3 PDDL-Driven Intelligent Adversary

To implement a goal-oriented adversary, we integrated an external automated planner using the Planning Domain Definition Language (PDDL)[4]. The adversary_domain.pddl file defines the universal predicates and actions available to the EX, such as move-to and chase-to-player, while problem files are dynamically generated from the Prolog KB each turn. This design decouples the adversary's high-level objectives from its low-level actions, creating a modular and extensible AI architecture. A core feature of the adversary is its dynamic plan evolution, which is managed through an internal finite-state machine driven by the ex_mood/1 fact. As shown in the Table 1, the adversary's behavior and PDDL goals change in direct response to the player's progress, ensuring its strategy adapts to counter the player. For instance, after the player collects the key, the adversary becomes angry, and its PDDL goal shifts to (at ex near-exit), compelling it to block the escape route. The system's plan execution and validation layer is a critical component for ensuring robustness. Prolog integrates with the STRIPS planner, Pyperplan, via a system call, parsing the resulting action sequence. Crucially, before each planned action is executed, its legality is verified against the current Prolog KB, for example by checking spatial reachability with connected/3. This acts as a vital bridge between the planner's abstract model and the game's real-time state. To handle cases of planning failure or illegal actions, we implemented a robust fallback strategy. If the planner returns "no solution" or an action is invalidated, the system gracefully reverts to a simpler, rule-based behavior defined in Prolog, such as random movement. This tiered AI architecture guarantees that the adversary always performs a valid action, ensuring system stability while enabling complex, goal-oriented behavior.

Table 1: Adversary States, Triggers, and PDDL Goals

State	Trigger	Behavior & PDDL Goal
Calm	Default	Random movement
Avoid	Player picks up the hat.	50% probability of not moving, 50% probability of random move
Angry	Player picks up the key.	Random movement, but tries to block exits
Chaser	Player picks up suitcase.	EX follows the player
Emotional	Player uses the photo.	EX approaches player to trigger "reunion ending"

4 List of Technical Implementations

The project integrates a range of technical features to create its dynamic and interactive experience. At its core is a dynamic Prolog knowledge base that manages real-time game state, including player and NPC locations, inventory, and mental health, using assert/1 and retract/1 for state updates. This is complemented by a rule-based room event system, where entering specific rooms triggers events like mental value deductions or narrative descriptions, linking the spatial environment to the game's psychological dimension. The adversary's intelligence is powered by an emotion-driven finite-state machine, where the ex_mood fact dictates its overall strategy, dynamically shifting from passive to aggressive based on player actions. This state machine controls a pluggable PDDL planning interface, which calls an external planner to generate goal-oriented action sequences. To ensure stability, a crucial plan action validation layer was implemented in Prolog to verify the legality of each planned action against the current game state before execution. To handle planning failures and increase unpredictability, the system incorporates a fallback strategy and utilizes randomness for adversary movement and capture probabilities. Finally, the game's narrative is enriched by a multiple-ending structure, offering four distinct outcomes (escape, capture, mental collapse, and reconciliation) based on the player's choices and performance.

5 Conclusion

This project successfully implements a sophisticated text-based adventure game by integrating two powerful symbolic AI paradigms: logic programming in Prolog and automated planning with PDDL. The resulting system effectively demonstrates a range of key competencies central to the study of knowledge representation, including static and dynamic fact modelling, rule-based inference, handling uncertainty, and orchestrating goal-oriented behaviour. The most significant technical challenge encountered during development was ensuring the perfect and persistent state synchronisation between the Prolog knowledge base and the PDDL problem instances, while also guaranteeing the logical consistency of the planner's output within the game world. This challenge was effectively surmounted through the implementation of a dedicated action validation layer and a robust, tiered fallback strategy. This design ensures that the system remains stable and responsive even when faced with planning anomalies. Through this blend of mechanics and narrative, we hope to convey a meaningful message about the courage required to leave an unhealthy relationship and embrace the possibility of a new life. Future work could productively explore the integration of more advanced AI techniques, such as extending the parser to handle more natural language inputs or incorporating more complex behavioural models to enhance the adversary's "believability"—the capacity to act in a way that is perceived as credible and intelligent by a human player[3]. (Word Count: 1441).

References

[1] Ronald J. Brachman and Hector J. Levesque. 1985. A Fundamental Tradeoff in Knowledge Representation and Reasoning (Revised Version). In *Readings in Knowledge Representation*. Morgan Kaufmann, 41–70.

[2] Zoya Z. C. Kocurek. 2020. Fabricating Affect: The Queer Potentials of Otome Games. *Game Studies* 20, 3 (2020).

[3] Michael Mateas and Andrew Stern. 2002. A Behavior Language for Believable Agents. In *AI and Interactive Entertainment: Papers from the 2002 AAAI Spring Symposium*. AAAI Press, 67–74.

[4] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. 1998. *PDDL—The Planning Domain Definition Language*. Technical Report CVC-98-003/DCS-TR-1165. Yale Center for Communicational Vision and Control.

[5] Georgios N Yannakakis and Julian Togelius. 2018. *Artificial Intelligence and Games*. Springer. doi:10.1007/978-3-319-63519-4